

NetComms 2 Module 3 Notes: Software Development and Design

DevNet Associate, Module 3. Built from DEVASC_Module_3. Read top to bottom; the blue boxes are memory aids I made (they are not from the slides).

How To Use These Notes

Module 3 has six topics: **3.1 Software Development, 3.2 Design Patterns, 3.3 Version Control (Git), 3.4 Coding Basics, 3.5 Code Review and Testing, 3.6 Data Formats**. Git commands and the three data formats are the usual exam targets, so spend the most time on those.

3.1 Software Development

The Software Development Life Cycle (SDLC)

SDLC is the process of building software **from an idea to delivery**. It is more than coding: it also includes gathering requirements, making a proof of concept, testing, and fixing bugs. It has **six phases**, and **each phase takes input from the result of the previous one**.

#	Phase	What Happens	Output
1	Requirements and Analysis	Explore stakeholders' situation, needs, constraints, and infrastructure. Analyze: is it possible and on budget? Risks to the schedule? How will it be tested? When and how is it delivered?	SRS (Software Requirement Specification) in classic waterfall, confirmed with stakeholders
2	Design	Architects and developers design the software from the SRS	HLD (High-Level Design) and LLD (Low-Level Design) documents
3	Implementation	Also called the coding or development phase. All components and modules are built. The longest phase.	Functional code implementing all requirements, ready to test
4	Testing	Code is installed in a test environment. Functional, integration, performance, and security testing. Continues until the code is bug free.	High-quality working software ready for production
5	Deployment	Software is installed into the production environment	Product manager releases the final software to end users
6	Maintenance	Support customers, fix bugs found in production, improve the software, gather	Team starts the next iteration and version

		new requests	
--	--	--------------	--

Memory Aid

Sentence: **Real Dogs Invite Tiny Dragons, Mostly.**

= **Requirements, Design, Implementation, Testing, Deployment, Maintenance.**

Output pairs: Requirements gives **SRS**. Design gives **HLD + LLD**. Implementation is the **longest**.

SRS = what to build. HLD = big picture. LLD = detail.

Three Popular Methodologies

A software development methodology is also called an **SDLC model**. The three most popular are **Waterfall, Agile, and Lean**. Which one to use depends on the **type of project, length of project, and size of the team**.

Model	Key Facts From The Slides
Waterfall	Original model created by Winston W. Royce ; the original had seven phases (shown as a diagram on the slide). Still widely used but gradually being replaced by more adaptive methods.
Agile	Flexible and customer-focused. A group of 17 developers wrote the Agile Manifesto in 2001 .
Lean	Based on Lean Manufacturing : minimize waste, maximize value to the customer.

Agile

The Four Values

- **Individuals and interactions** over processes and tools
- **Working software** over comprehensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a plan

Memory Aid

Left side only: **People, Product, Partners, Pivot.**

The Twelve Principles (Cisco Keywords)

Customer focus; Collaboration; Working software; Simplicity; Embrace change and adapt; Motivated teams; Work at a sustainable pace; Self-organizing teams; Frequent delivery of working software; Face-to-face conversations; Agile environment; Continuous improvement.

Popular Agile Methods

Method	Focus
Agile Scrum	Small, self-organizing teams that meet daily for short periods

	and work in iterative sprints
Lean	Eliminate wasted effort in planning and execution; reduce programmer cognitive load
Extreme Programming (XP)	Addresses the quality-of-life issues of development teams
Feature-Driven Development (FDD)	Work from an overall model, then break out, plan, design, and build feature by feature

Memory Aid

Four Agile methods: **S-L-X-F** = Scrum, Lean, XP, FDD. "Some **L**ions **X**-ray **F**ish."

Sprints, Backlog, And User Stories

- **Sprint:** a fixed period, usually **2 to 4 weeks**. The team takes as many tasks (user stories) as it can finish. At the end the software should be **working and deliverable**. Length is set before starting and rarely changes.
- **Backlog:** all the features of the software in a **prioritized list**.
- **User story:** a simple statement of what a user (or role) needs, and why. Small enough for one team to finish in one sprint.
- **Scrum team:** cross-functional, collaborative, self-managed, **no more than 10 people**. The scrum master holds a **daily stand-up**, same time every day, **no more than 15 minutes**: what is finished, in progress, or about to start.

User Story Template

As a <user or role>, I would like to <action>, so that <value or benefit>.

Memory Aid

Template = **Who, What, Why**. As a WHO, I want WHAT, so that WHY.

Lean: Seven Principles (Cisco Wording)

Principle	Meaning
Eliminate waste	The most fundamental lean principle
Amplify learning	Short sprints of working software: developers learn faster, customers give feedback sooner, features are adjusted to bring more value
Decide as late as possible	Under uncertainty, delay decisions so they rest on facts, not opinions or speculation
Deliver as fast as possible	Enables feedback, amplifies learning, makes decisions faster, gives required features, produces less waste. It does not allow customers to change their mind.
Empower the team	Each person decides in their own area of expertise
Build integrity in	The software addresses the customer's needs and stays useful to them
Optimize the whole	Build cohesively; if each expert looks only at their own part,

	the value of the whole suffers
--	--------------------------------

Memory Aid

Even A Dull Day Earns Bright Optimism = Eliminate, Amplify, Decide late, Deliver fast, Empower, Build integrity in, Optimize the whole.

The Seven Wastes

- Partially done work
- Extra processes
- Extra features
- Task switching
- Waiting
- Motion
- Defects

Memory Aid

Please Eat Every Tasty Waffle, My Darling = Partial work, Extra processes, Extra features, Task switching, Waiting, Motion, Defects.

3.2 Software Design Patterns

Design patterns are **best-practice solutions to common problems** and are **language independent**. In 1994, **Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides** (the **Gang of Four, GoF**) published *Design Patterns: Elements of Reusable Object-Oriented Software*. Two key ideas from it: **program to an interface, not an implementation**, and **favor object composition over class inheritance**.

The GoF listed **23 patterns** in **three categories: Creational, Structural, Behavioral**. The two most common patterns named in the module are the **Observer** (a Behavioral pattern) and **MVC**.

Memory Aid

GoF categories: **Create, Structure, Behave** (Creational, Structural, Behavioral).

Gang of Four = **4 names, 23 patterns, 1994**.

Observer Pattern

A **subscription and notification** design: objects receive events when something they are observing changes. To work, the **subject** must **store a list of its observers** and have methods to **add and remove** observers. Benefit: observers get **real-time data** when a change happens. Subscription mechanisms perform better than alternatives such as **polling**.

Memory Aid

Observer = **YouTube subscribe + bell**. You do not keep checking (polling). The channel (subject) keeps a list of subscribers and notifies them.

Model-View-Controller (MVC)

MVC simplifies apps that depend on **graphical user interfaces** by splitting code and responsibility into three parts. Benefit: each component can be **built in parallel**.

Part	Job
Model	The application's data structure . Manages the data, logic, and rules . Gets input from the controller.
View	The visual representation of the data
Controller	The middleman between model and view. Takes in user input and manipulates it to fit the format for the model or view.

Memory Aid

M = Memory (data and rules). **V = Visible** (what you see). **C = Connector** (carries user input between them).

Watch Out

The slide says the controller reshapes input "to fit the format for the model or view". On Dawn's quiz key the accepted answer for "the controller manipulates user input to fit the format for ____" was **the model**. Read the answer choices: if both "model" and "model or

view" appear, follow the quiz key (model).

3.3 Version Control Systems

Version control (also revision control or source control) manages changes to a set of files and keeps a history.

Five benefits: enables **collaboration, accountability and visibility, work in isolation, safety, work anywhere.**

Three Types

Type	Model	Key Facts
Local (LVCS)	A simple local database	Stores the delta between versions; reverse the delta to get an older version
Centralized (CVCS)	Server-client ; repository on a central server	Only one person edits a file at a time. Check out to lock , check in when done
Distributed (DVCS)	Peer-to-peer ; repository usually on a hosting service	Everyone works on any file at the same time, no locking . Push to the main repo; the system detects conflicts

Memory Aid

Local = on my laptop only. Centralized = one Chair, one person at a time (lock). Distributed = Dance floor, everyone at once (no lock).

Git

Git is an **open source DVCS**. Key difference from other systems: Git stores **snapshots, not differences (deltas)**. If a file did not change, Git stores a **reference link** to the last snapshot instead of a new copy.

Three Stages And Three States

Three Stages (Places)	Three States
Repository (the .git directory)	Committed
Working directory	Modified
Staging area	Staged

Memory Aid

A file travels: **Modified** (in the working directory) then **Staged** (in the staging area) then **Committed** (in the repository).

Words: **Work, Stage, Store**. Edit it, stage it, store it.

Local Vs Remote Repositories

- **Local:** on your own machine, where you run the git commands.
- **Remote:** somewhere else, usually a server or hosting service. It still counts as DVCS because it holds the **full repository (code and history)**.
- Cloning gets the full repository **without locking**.
- After cloning (or creating the remote from local), the two are **independent** until you apply changes with a **manual Git command**.

Git Vs GitHub

Git is the distributed version control implementation with a command-line interface. **GitHub** is a **service provided by Microsoft** that hosts repositories with Git and adds code review, documentation, project management, bug tracking, and feature requests. GitHub introduced the **pull request**: a formal request to review a contributor's branch changes for inclusion in the main branch.

Git Commands

Command	What It Does
git config --global key value	Sets the initial global settings
git init <project directory>	Creates an empty repository or makes an existing folder one. Creates a hidden .git folder (metadata, compressed files, commit history, staging area) and the master branch.
git clone <repository> [target directory]	Gets an existing repository. Four transport protocols: Local, SSH, Git, HTTP .
git status	Lists files that differ between the working directory and the parent branch
git diff	Generic file comparison; the file does not need to be tracked by Git
git add <file>	Adds files to the staging area . git add . adds all changed files. Only the files you name are added.
git rm <file>	Removes the file from the working directory and stages the removal. Does not work if the file is already staged with changes.
git rm --cached <file>	Removes from the staging area without deleting the file from the working directory
git commit -m "<message>"	Combines the staged changes into one commit and updates the local repository
git push origin <branch>	Sends local commits to a branch of the remote. Fails if there is a conflict . git push origin master for master.
git pull or git pull origin <branch>	Manually updates your local copy from the remote (local copies do not update automatically)
git branch	Lists local branches (also git branch --list). Also creates branches.
git checkout -b <branch>	Creates and switches to a branch

git branch -d <branch>	Deletes a branch
git merge <branch>	Merges a branch into the current branch. Several branches can be listed.

Memory Aid

Edit, Add, Commit, Push. Local work moves **add** (to stage) then **commit** (to local repo) then **push** (to remote). **Pull** is the reverse direction.

Push = up, Pull = down.

-d deletes a branch; **-b** in checkout means **build** a new branch and switch.

What git pull Does (Four Steps)

1. The local repository (.git) is updated with the latest commits and history from the remote.
2. The working directory and branch are updated with that content.
3. A single commit is created on the local branch with those changes.
4. The working directory is updated with the latest content.

Branching And Merging

- Branching lets you work **independently** without affecting the main code. A new repository starts on a branch called **Master**.
- Branches can be local or remote, deleted, and each has its **own history, staging area, and working directory**. Creating and switching is **lightweight and almost instant**.
- Switching branches changes the working directory and staging area but **not** the .git repository.
- In a merge, **only the target branch changes**. The **source branch stays the same**.
- **Fast-forward merge:** Git applies the source commits to the target **automatically, without conflicts**.
- **Merge conflict:** Git cannot fast-forward because it does not know how to combine the changes to the file(s).

Reading A .diff File

Symbol	Meaning
+	The line was added
-	The line was removed
/dev/null	A file was added or removed
blank (space)	Context lines around the changes
@@	Marks that the next block of changes is starting (a file can have several)
index	Shows the commits compared

Memory Aid

Plus = Put in, Minus = Missing now.

3.4 Coding Basics

Clean Code

Clean code is written so **other developers can read and understand it**. It follows common principles on formatting, organization, intuitive components, purpose, and reusability, and stresses **standardization, proper organization, modularity, and inline comments** so it is self-documenting.

Why write it: easier to understand, more compact and organized; modular code is easier to **unit test**; standardized code is easier to **scan with automated tools**; and it looks nicer.

Functions, Methods, Modules, Classes

Term	Meaning
Function	A standalone block of code that performs a task when executed. Write once, run many times. Code that does a discrete task, or is used more than once, is a candidate to encapsulate.
Method	A block of code associated with an object , typically in object-oriented programming
Parameters and arguments	Add flexibility to functions. Parameters are the placeholders in the definition; arguments are the values you pass in.
return	Ends the function and sends a value back to the caller. Any code below the return is skipped.
Module	A set of functions packaged as a single file , expected to work independently, with an interface for other modules. Used to divide a large project (example: circleClass.py).
Class	Bundles data and functionality . Each class declaration defines a new object type . Can be instantiated many times, each with its own attribute values, and can inherit from existing classes.

Memory Aid

Function = free agent. Method = belongs to an object. "Methods are attached to a class."

Watch Out

Python has **no true private** variables or methods. By convention, a name with a **single leading underscore** (`_`) is treated as private.

3.5 Code Review And Testing

Code Review

Developers (reviewers) look over the codebase, part of it, or specific changes and give feedback. It happens **only after the code changes are complete and tested**. Goal: the final code is easy to

read and understand, follows best practices, is correctly formatted, is free of bugs, has proper comments and documentation, and is clean.

Type	How It Works
Formal code review	A series of meetings to review the whole codebase
Change-based code review	Also called tool-assisted . Reviews code changed because of a bug, user story, feature, commit, etc.
Over-the-shoulder code review	A reviewer looks over the shoulder of the developer who wrote it and gives feedback
Email pass-around	Happens after automatic emails from the source control system when a check-in is made

Memory Aid

F-C-O-E: Formal (meetings, whole code), Change-based (tool-assisted), Over-the-shoulder (live, beside you), Email (auto email on check-in).

Testing

- **Functional testing:** does the software work correctly and behave as intended? From **unit testing** (lowest level) to **integration testing** (higher level).
- **Non-functional testing:** usability, performance, security, resiliency, compliance, localization. Is the software fit for purpose?
- **Unit testing:** detailed functional tests of small pieces (lines, blocks, functions, classes) **in isolation**.
- **Integration testing:** makes sure the individual units **fit together** into a complete application.

Python Test Framework	How It Works
PyTest	Automatically runs scripts that start with test_ or end with _test.py , and within them runs functions beginning with test_
unittest	Different syntax: you subclass the built-in TestCase class and add methods whose names begin with test_

Test-Driven Development (TDD)

Developers capture design requirements as **tests first**, then write software to pass them. The pattern is a **five-step repeating loop**:

1. Create a new test.
2. Run the tests to see if any fail for unexpected reasons.
3. Write application code to pass the new test.
4. Run the tests to see if any fail.
5. Refactor and improve the application code.

Memory Aid

T-R-W-R-R: Test, Run, Write, Run, Refactor. Test comes **before** code. Refactor is always last.

3.6 Understanding Data Formats

REST APIs exchange information with remote services. The **three most popular formats** are **XML, JSON, and YAML**. A common REST pattern has four steps:

1. **Authenticate**, usually by POSTing a user and password and getting back an **expiring token**.
2. **GET** a resource from an endpoint, asking for XML, JSON, or YAML.
3. **Modify** the returned data.
4. **POST (or PUT)** to the same endpoint to change the resource's state, and read the reply to see if it worked.

Compare The Three

Point	XML	JSON	YAML
Stands for	Extensible Markup Language	JavaScript Object Notation	YAML Ain't Markup Language
Family	Derived from SGML ; parent of HTML	Derived from how JavaScript writes object literals	A superset of JSON
File ending	.xml	.json	.yaml or .yml
Structure	Symmetrical tags , names are user-defined	Key/value pairs in braces; arrays in brackets	Indentation ; list items start with a dash and space
Comments	Yes (same as HTML)	No standard comments	Yes , starts with # and a space
Whitespace		Not significant	Significant (shows hierarchy)
Types		Numbers, strings, Booleans, nulls	Numbers, strings, Booleans, nulls

Memory Aid

JSON has no comments. XML and YAML do.

YAML can parse JSON, but JSON cannot parse YAML (**superset**).

YAML files open with --- and end with

XML Extras

- **Prologue**: the first line of the file.
- **Attributes**: extra information placed inside a tag.
- **Namespaces**: identified by URIs, set with the **xmlns** attribute (example on the slide: NETCONF 1.0).
- Special characters are encoded because data is readable text.

Parsing Vs Serializing

Term	Meaning
Parsing	Analyze a message, break it into parts, and understand their purpose.

	Turns XML, JSON, or YAML text into a data structure.
Serializing	Roughly the opposite: turns an internal data structure into a formatted character string for sending.

Memory Aid

Parse = Pull apart. Serialize = Send out.

One-Page Cram Sheet

Topic	Remember
SDLC phases	Requirements, Design, Implementation (longest), Testing, Deployment, Maintenance
SRS / HLD / LLD	SRS after requirements; HLD and LLD after design
Methodologies	Waterfall (Royce), Agile (17 developers, 2001), Lean
Agile methods	Scrum, Lean, XP, FDD
Sprint / team / stand-up	2 to 4 weeks / max 10 people / max 15 minutes
User story	As a ___, I would like to ___, so that ___
GoF	4 authors, 1994, 23 patterns, Creational/Structural/Behavioral
Observer	Subject keeps observer list; add and remove; better than polling
MVC	Model = data and rules; View = visual; Controller = middleman
VCS types	Local (delta), Centralized (lock), Distributed (no lock)
Git	Snapshots not deltas; Working directory, Staging area, Repository
Git states	Modified, Staged, Committed
Git flow	add, commit, push; pull to update
Merge	Only target changes; conflict means no fast-forward
Code review types	Formal, Change-based, Over-the-shoulder, Email pass-around
TDD	Test, Run, Write, Run, Refactor
Data formats	XML (tags), JSON (no comments), YAML (superset of JSON, indentation)